Software Requirements Specification

AzDiag

Version 1.0

Author: Joey Han

Date: October 2026

Table of Contents

1. Introduction 3
    1.1 Purpose 3
    1.2 Scope 3
    1.3 Definitions 3
    1.4 References 3
2. Overall Description 4
    2.1 Product Perspective 4
    2.2 User Classes 4
    2.3 Operating Environment 4
    2.4 Constraints 4
    2.5 Assumptions 4
3. Functional Requirements 4
    3.1 Authentication and Configuration 5
    3.2 Telemetry Collection 5
    3.3 Analysis 5
    3.4 LLM Explanation 5
    3.5 User Interface (Web UI) 6
    3.6 Logging and Export 6
4. Non-Functional Requirements 7
    4.1 Performance 7
    4.2 Security 7
    4.3 Reliability and Accuracy 7
    4.4 Cost 7
    4.5 Maintainability and Portability 8
    4.6 Usability 8
5. External Interface Requirements 8
6. Use Cases 9
    UC-1: Diagnose a resource 9
    UC-2: Ask a follow-up question (optional) 9
7. Acceptance Criteria 9
8. Future Enhancements (Out of scope for V1.0) 9

1. Introduction

1.1 Purpose

This document specifies the requirements for Azure Diagnostic Bot (AzDiag) which is a Python application that reads live telemetry from Microsoft Azure resources and uses a locally hosted LLM to explain in plain English what is wrong and what to check next.

1.2 Scope

AzDiag will:

- Pull metrics and logs from Azure App Service resources (optional: active alerts)
- Detect anomalies using simple rule-based checks
- Send a condensed summary of the findings to a local LLM
- Return a plain-English diagnosis with suggested next steps
- Present the diagnosis in a simple local web UI

AzDiag will not automatically change, restart or delete any Azure resources (read-only). This is a learning project and not a production monitoring tool.

1.3 Definitions

Term	Meaning
Telemetry	Metrics, logs and traces emitted by Azure resources
Azure Monitor	Azure's service for collecting metrics and logs
Log Analytics	Azure's log store, queried with KQL
KQL	Kusto Query Language
LLM	Large Language Model (default: qwen3.5:4b via Ollama)
Ollama	Tool for running LLMs locally, exposes a local REST API
Streamlit	Open-source Python framework for building simple web UIs
Web UI	Browser-based interface served locally by AzDiag for running and reading diagnoses
PII (Personally Identifiable Information)	A set of data that could be used to distinguish a specific individual

1.4 References

- Azure Monitor documentation
- Azure SDK for Python (azure-monitor-query, azure-identity)
- Ollama API documentation
- Streamlit documentation

2. Overall Description

2.1 Product Perspective

AzDiag is a standalone application with four layers: a telemetry collector (Azure), an analyser (Python rules), an explainer (local LLM) and a web UI (Streamlit). It runs on the developer's machine and talks to Azure over HTTPS.

```
Azure Resources -> Azure Monitor / Log Analytics -> Collector -> Analyser
-> LLM (Ollama) -> Web UI (Streamlit) -> Plain-English Report
```

2.2 User Classes

- Primary user: A developer or learner who wants quick and readable explanations of Azure problems.

2.3 Operating Environment

- Python 3.11+
- Windows, macOS or Linux
- Ollama running locally with qwen3.5:4b
- A modern web browser to open the local web UI (Google Chrome, Firefox)
- Azure Free subscription

2.4 Constraints

- C-1: Must operate within Azure Free subscription limits (including the Log Analytics free ingestion allowance and the credit/free-service periods). Verify current limits on Azure's pricing pages before building.
- C-2: A 4B-parameter model has limited reasoning ability and a limited effective context window, so raw telemetry must be summarised before being sent to it.
- C-3: The LLM runs locally, so response time depends on the user's hardware.
- C-4: No secrets in source control.

2.5 Assumptions

- The user has Azure CLI installed and can sign in
- At least one monitored resource exists (example: an Azure App Service web app)
- The local machine has enough RAM to run a 4B model

3. Functional Requirements

Priorities use the MoSCoW scheme:

Must = required for the project to be considered complete

Should = important but not essential

Could = optional

3.1 Authentication and Configuration

ID	Requirement	Priority
FR-1	The system shall authenticate to Azure using DefaultAzureCredential (no hardcoded keys).	Must
FR-2	The system shall read configuration (subscription ID, resource IDs, workspace ID, model name) from a .env or YAML config file.	Must
FR-3	The system shall validate configuration at startup and report missing values clearly.	Should

3.2 Telemetry Collection

ID	Requirement	Priority
FR-4	The system shall retrieve platform metrics (example: CPU %, memory, HTTP 5xx count, response time) for configured resources via Azure Monitor.	Must
FR-5	The system shall run predefined KQL queries loaded via an external YAML configuration file against a Log Analytics workspace to retrieve recent errors and exceptions.	Must
FR-6	The system shall support a configurable time window (default: last 60 minutes).	Must
FR-7	The system shall retrieve active Azure Monitor alerts for the subscription or resource group.	Could
FR-8	The system shall handle API errors, throttling (HTTP 429) and empty results without crashing, using exponential backoff for transient errors.	Must

3.3 Analysis

ID	Requirement	Priority
FR-9	The system shall apply threshold-based rules (example: CPU > 85% for 10 min, 5xx rate > 5%) to detect anomalies.	Must
FR-10	The system shall group repeated log errors and rank them by frequency.	Should
FR-11	The system shall produce a compact structured summary of findings (JSON), limited to a configurable size (example: 2,000 tokens) before sending to the LLM. Log snippets shall be truncated to a maximum character or token ceiling to prevent context-window overflow on small models.	Must

3.4 LLM Explanation

ID	Requirement	Priority
FR-12	The system shall send the structured summary to qwen3.5:4b through the Ollama local API.	Must
FR-13	The prompt shall instruct the model to produce: (a) a one-sentence summary, (b) likely causes, (c) suggested next steps, (d) a confidence level.	Must

ID	Requirement	Priority
FR-14	The prompt shall instruct the model to rely only on the supplied data and say "not enough information" when uncertain.	Must
FR-15	The system shall request structured output (JSON schema) and validate it, retrying once with a corrective prompt on invalid output and falling back to raw text display if validation fails a second time.	Must
FR-16	The system shall allow the model's reasoning ("thinking") mode to be switched on or off by configuration.	Could

3.5 User Interface (Web UI)

ID	Requirement	Priority
FR-17	The system shall provide a simple local web UI built with Streamlit, started with streamlit run app.py and opened in a browser.	Must
FR-18	The UI shall let the user select a configured resource and a time window (default: 60 minutes) and start a diagnosis with a single "Diagnose" button.	Must
FR-19	The UI shall display the diagnosis in a readable format (summary, likely causes, next steps, confidence) with the raw evidence (metrics and log snippet) in expandable sections beneath it.	Must
FR-20	The UI shall show progress (spinner or status messages) during telemetry collection and LLM inference and show clear error messages if Azure or Ollama is unavailable.	Should
FR-21	The UI shall provide a chat box for follow-up questions (example: "Why is CPU high?") answered from the current diagnosis.	Could
FR-22	The UI shall provide an auto-refresh toggle that re-runs the diagnosis at a set interval.	Could
FR-23	The UI shall display a "healthy" status together with key metrics when no anomalies are detected and shall not report a problem that the analyser did not flag.	Must

3.6 Logging and Export

ID	Requirement	Priority
FR-24	The system shall log its own activity (queries run, durations and errors) to a local file.	Should
FR-25	The UI shall let the user download each diagnosis report as Markdown or JSON.	Could

4. Non-Functional Requirements

In this section, Must and Should have the same meaning as in section 3.

Target marks a measurable goal rather than a strict requirement.

4.1 Performance

ID	Requirement	Type
NFR-1	Telemetry collection shall complete within 15 seconds for a single resource under normal conditions.	Target
NFR-2	LLM response shall be returned within 60 seconds on a machine with 16 GB RAM.	Target

4.2 Security

ID	Requirement	Type
NFR-3	The system shall use read-only Azure roles (example: Monitoring Reader, Log Analytics Reader).	Must
NFR-4	Credentials and IDs shall not be committed to the repository (.gitignore for .env).	Must
NFR-5	Telemetry and diagnoses shall not be sent to any service other than Azure and the local Ollama instance. The LLM is local, so no data is sent to third-party AI services.	Must
NFR-6	The system shall pass telemetry and log snippets through a sanitisation filter using regex patterns to redact potential secrets (example: connection strings, bearer tokens, API keys, emails and explicit user PII) before constructing the payload for the local LLM.	Must
NFR-7	The web UI shall bind to localhost only and shall not be exposed to the network by default.	Must

4.3 Reliability and Accuracy

ID	Requirement	Type
NFR-8	The system shall fail gracefully if Azure or Ollama is unreachable, showing a clear error message.	Must
NFR-9	Every claim in the diagnosis shall be traceable to evidence shown in the report.	Must
NFR-10	The report shall display a disclaimer that the output is AI-generated and may be wrong.	Must

4.4 Cost

ID	Requirement	Type
NFR-11	The system shall run with no paid services. Query frequency shall be limited to stay within free-tier quotas.	Must

4.5 Maintainability and Portability

ID	Requirement	Type
NFR-12	Code shall be modular (collector, analyser, LLM client, web UI) with type hints and docstrings.	Should
NFR-13	Unit test coverage of the analyser and prompt-building logic shall be at least 70%.	Should
NFR-14	The LLM client shall be swappable (the model name is configurable).	Should

4.6 Usability

ID	Requirement	Type
NFR-15	A new user shall be able to set up and run the first diagnosis in under 30 minutes using the README.	Target
NFR-16	Each anomaly in the diagnosis shall include a plain-language explanation and Azure-specific terms shall be briefly explained or avoided.	Should
NFR-17	A user shall be able to start a diagnosis from the web UI in three clicks or fewer, without reading documentation.	Should

5. External Interface Requirements

Interface	Details
Azure Monitor Metrics API	via azure-monitor-query MetricsQueryClient
Log Analytics Query API	via LogsQueryClient, KQL queries
Azure Identity	azure-identity (DefaultAzureCredential)
Ollama REST API	http://localhost:11434/api/chat
Web browser (Streamlit)	Local web UI served at http://localhost:8501
Filesystem	Config file, log file and report output folder

6. Use Cases

UC-1: Diagnose a resource

- **Actor:** Developer
- **Precondition:** Azure login active, Ollama running and web app started
- **Main flow:**
    a. User opens the web UI, selects my-webapp and a time window, and clicks Diagnose.
    b. The system collects metrics and logs.
    c. Analyser flags anomalies.
    d. Summary is sent to the LLM.
    e. The web UI displays the diagnosis with the evidence in expandable sections beneath it.
- **Alternate flow:** No anomalies found. The system reports "healthy" and shows key metrics.
- **Exception:** Azure auth fails. The UI shows an error message with instructions to run az login.

UC-2: Ask a follow-up question (optional)

The user types "Which errors are most frequent?" into the chat box and the system answers from the cached summary.

7. Acceptance Criteria

1. Running a diagnosis from the web UI against a deliberately stressed test resource produces a diagnosis that correctly identifies the injected problem (example: high CPU) in at least 4 of 5 trials. (Covers FR-4, FR-9, FR-13, FR-19)
2. When no anomaly exists, the system does not invent a problem. (Covers FR-23)
3. The system runs end to end on the Azure Free subscription with no charges. (Covers NFR-11)
4. Telemetry collection completes within 15 seconds and the LLM response within 60 seconds on a machine with 16 GB RAM. (Covers NFR-1, NFR-2)
5. A fake API key planted in the logs does not appear in the prompt sent to the LLM. (Covers NFR-6)
6. All Must-priority requirements are implemented and tested.

8. Future Enhancements (Out of scope for V1.0)

- Other Microsoft Azure resources (example: Azure Virtual Machines (VMs) and Azure SQL Database)